SPECULATIVE DECODING WITH TRANSFORMER LLMS: COMPLETE CODE & MATH EXPLANATION

MODULE 1: PROJECT OVERVIEW & MATHEMATICAL FOUNDATION

WHAT WE'RE BUILDING

A complete inference optimization system for LLMs: - **Target Model:** 10.8M parameter decoder-only Transformer (the "good" model) - **Draft Model:** 1.0M parameter lightweight Transformer (the "fast" model) - **Speculative Decoding:** Uses the small model to guess multiple tokens, then verifies them in parallel with the big model

Result: ~2.1x faster inference with *mathematically identical* output distribution.

THE CORE PROBLEM: MEMORY BANDWIDTH BOTTLENECK

Autoregressive generation is **sequential**: to generate token `t+1`, you need token `t`. Each forward pass through a 10.8M model loads all weights from GPU memory (HBM) even though you only produce one token.

Math: - Compute per token: ~2 × 10.8M = 21.6M FLOPs - Memory per token: ~10.8M weights × 4 bytes = 43.2 MB - Arithmetic intensity: 21.6M / 43.2M = 0.5 FLOPs/byte

GPUs need ~10+ FLOPs/byte to be compute-bound. We're heavily **memory-bound** — the GPU is idle while waiting for weights.

Solution: Generate multiple tokens per weight load. That's what speculative decoding does.

MODULE 2: THE SPECULATIVE DECODING ALGORITHM (LEVIATHAN ET AL., 2023)

STEP 1: DRAFT GENERATION

Generate `γ` candidate tokens from the draft model `M_draft`:

[CODE // CODE] >> SOURCE BUFFER

$$\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_\gamma \sim M_{draft}(\cdot \mid x_{1:N})$$

Math: For each of `γ` steps, sample from `p_draft(x | context)`. This is fast because the draft model is ~10x smaller.

STEP 2: PARALLEL VERIFICATION

Evaluate ALL `γ` candidates in **one forward pass** of the target model:

[CODE // CODE] >> SOURCE BUFFER

$$q_i = M_{target}(\cdot \mid x_{1:N}, \tilde{x}_{1:i-1})$$

Math: Because the Transformer is causal, feeding `[x₁:N, x̃₁, ..., x̃_γ]` gives us the target distribution for each position simultaneously. One matmul for `γ+1` predictions.

STEP 3: REJECTION SAMPLING (THE ACCEPTANCE TEST)

For each candidate $\tilde{x}_i$, accept with probability:

[CODE // CODE] >> SOURCE BUFFER

$$\alpha_i = \min(1, q_i(\tilde{x}_i) / p_i(\tilde{x}_i))$$

Math: This is rejection sampling. If the target likes the draft's choice (q > p), always accept. If not, accept with probability q/p.

Why this works: The acceptance probability is chosen so the marginal distribution of accepted tokens equals the target distribution q.

STEP 4: RESIDUAL RESAMPLING (ON REJECTION)

If rejected, sample from the **normalized positive residual**:

[CODE // CODE] >> SOURCE BUFFER

$$x_i \sim \max(0, q_i(x) - p_i(x)) / \Sigma_y \max(0, q_i(y) - p_i(y))$$

Math: $q - p$ is the "excess" probability mass the target has over the draft. Sampling from this corrected distribution ensures the final output is exactly q.

Proof sketch:

[CODE // CODE] >> SOURCE BUFFER

$$P(\text{accept } x) = p(x) \cdot \min(1, q(x)/p(x)) = \min(p(x), q(x))$$
$$P(\text{reject, then sample } x) = (1 - \Sigma \min(p,q)) \cdot \max(0, q(x) - p(x)) / \Sigma \max(0, q-p)$$
$$= \max(0, q(x) - p(x))$$
$$\text{Total: } \min(p(x), q(x)) + \max(0, q(x) - p(x)) = q(x) \checkmark$$

This is the key theorem: Speculative decoding produces the exact same distribution as the target model alone. It's a lossless speedup.

MODULE 3: THE TRANSFORMER ARCHITECTURE (FROM SCRATCH)

3.1 CONFIGURATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
@dataclass
class GPTConfig:
    block_size: int = 256      # max context length
    vocab_size: int = 10000    # BPE vocab
    n_layer: int = 9           # number of transformer blocks
    n_head: int = 4            # attention heads
    n_embd: int = 272          # embedding dimension
    dropout: float = 0.0
    bias: bool = True
    tie_weights: bool = True
```

Parameter count formula:

[CODE // CODE] >> SOURCE BUFFER

```
Params ≈ vocab_size × n_embd × 2 (if tied) + n_layer × (12 × n_embd² + 13 × n_embd)
```

For target: `10000 × 272 + 9 × (12 × 272² + 13 × 272) ≈ 10.8M` ✓

3.2 LAYERNORM

[CODE // PYTHON] >> SOURCE BUFFER

```python
class LayerNorm(nn.Module):
    def __init__(self, ndim, bias):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(ndim))
        self.bias = nn.Parameter(torch.zeros(ndim)) if bias else None

    def forward(self, input):
        return F.layer_norm(input, self.weight.shape, self.weight, self.bias, 1e-5)
```

Math: For each token's vector `x ∈ R^d` :

[CODE // CODE] >> SOURCE BUFFER

$$\mu = (1/d) \sum x_i$$
$$\sigma^2 = (1/d) \sum (x_i - \mu)^2$$
$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$$
$$\text{out} = \gamma \cdot \hat{x} + \beta$$

Why: Normalizes feature distribution per token, independent of batch. Stabilizes training.

3.3 CAUSAL SELF-ATTENTION (WITH KV-CACHE)

[CODE // PYTHON] >> SOURCE BUFFER

```python
class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        self.head_dim = config.n_embd // config.n_head
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=config.bias)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd, bias=config.bias)
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
```

Math of `c_attn`: A single Linear layer produces Q, K, V for all heads at once.

Input `x: (B, T, C)` → `x @ W_qkv^T` where `W_qkv: (3C, C)` → output `(B, T, 3C)`.

Then `.split(C, dim=2)` gives three `(B, T, C)` tensors.

FORWARD PASS

[CODE // PYTHON] >> SOURCE BUFFER

```python
def forward(self, x, past_kv=None, use_cache=False):
    B, T, C = x.size()
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)  # (B, H, T, D)
    k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
    v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
```

Math: Reshape `(B, T, C)` into `(B, T, H, D)` where `C = H × D`, then transpose to `(B, H, T, D)` for batched attention.

For target: `H=4, D=272/4=68`. For draft: `H=2, D=88/2=44`.

KV-CACHE CONCATENATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
if past_kv is not None:
    k_past, v_past = past_kv
    k = torch.cat([k_past, k], dim=2)
    v = torch.cat([v_past, v], dim=2)
```

Math: For autoregressive generation, K and V from previous timesteps don't change. We cache them and append new ones.

Without cache: Compute T^2 attention entries at each step → $O(T^2)$ total per token. **With cache:** Compute only T new attention entries per step → $O(T)$ per token.

ATTENTION WITH DIFFERENT MODES

```python
if past_kv is None and not use_cache:
    # Standard training: full causal attention via FlashAttention
    y = F.scaled_dot_product_attention(q, k, v, is_causal=(T > 1))
```

Math (FlashAttention): Computes `softmax(QK^T/√D) V` without materializing the `T×T` matrix in HBM. Uses online softmax + tiling.

```python
elif T == 1:
    # Single-token decode: no causal mask needed (query sees all keys)
    y = F.scaled_dot_product_attention(q, k, v, is_causal=False)
```

Math: When `T=1`, the single query attends to all `T_cache` keys. No mask needed since the query is at the latest position.

```python
else:
    # Speculative verification: explicit causal mask
    q_indices = torch.arange(past_len, total_kv_len).unsqueeze(1)
    k_indices = torch.arange(0, total_kv_len).unsqueeze(0)
    causal_mask = (k_indices <= q_indices).unsqueeze(0).unsqueeze(0)
    y = F.scaled_dot_product_attention(q, k, v, attn_mask=causal_mask, is_causal=False)
```

Math: For speculative decoding, we have `T > 1` new queries but existing cached keys. The mask must allow query `i` (absolute position `past_len+i`) to see keys `0..past_len+i`.

This is the trick that enables parallel verification.

3.4 MLP (FEED-FORWARD)

```python
class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=config.bias)
        self.gelu = nn.GELU()
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        return self.dropout(self.c_proj(self.gelu(self.c_fc(x))))
```

Math:

[CODE // CODE] >> SOURCE BUFFER

```
h = GELU(x W₁ + b₁)      # (B, T, 272) → (B, T, 1088)
out = h W₂ + b₂          # (B, T, 1088) → (B, T, 272)
```

GELU approximation: `GELU(x) ≈ 0.5x(1 + tanh(√(2/π)(x + 0.044715x³)))`

Why 4×: From original paper. Gives the network more capacity for nonlinear feature computation.

3.5 TRANSFORMER BLOCK

[CODE // PYTHON] >> SOURCE BUFFER

```python
class Block(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.mlp = MLP(config)

    def forward(self, x, past_kv=None, use_cache=False):
        attn_out, next_kv = self.attn(self.ln_1(x), past_kv=past_kv, use_cache=use_cache)
        x = x + attn_out          # Residual
        x = x + self.mlp(self.ln_2(x))  # Residual
        return x, next_kv
```

Math: Pre-norm formulation:

[CODE // CODE] >> SOURCE BUFFER

```
x' = x + Attn(LN(x))
x'' = x' + MLP(LN(x'))
```

Why residual: Gradient flows directly through the `+` operation: `∂(x + f(x))/∂x = 1 + ∂f/∂x`. The `1` prevents vanishing gradients in deep networks.

3.6 FULL GPT MODEL

[CODE // PYTHON] >> SOURCE BUFFER

```python
class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embd),   # token embeddings
            wpe = nn.Embedding(config.block_size, config.n_embd),   # position embeddings
            drop = nn.Dropout(config.dropout),
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f = LayerNorm(config.n_embd, bias=config.bias),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        if config.tie_weights:
            self.transformer.wte.weight = self.lm_head.weight
```

Math: Weight tying — the token embedding matrix `W_te: (V, C)` is the same as the transposed LM head `W_lm: (C, V)`. This: 1. Saves `V × C` parameters (for target: 10000 × 272 = 2.72M params) 2. Improves performance (both operations learn similar representations)

WEIGHT INITIALIZATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

Math: `W ~ N(0, 0.02²)`. Small values prevent softmax saturation at init.

Scaled residual init:

[CODE // PYTHON] >> SOURCE BUFFER

```python
for pn, p in self.named_parameters():
    if pn.endswith("c_proj.weight"):
        torch.nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 * config.n_layer))
```

Math: Output projections in residual branches are scaled by `1/√(2L)` where `L = n_layer`. This ensures the residual sum `x + Σ branch_i(x)` has bounded variance as depth grows.

FORWARD PASS WITH CACHE

[CODE // PYTHON] >> SOURCE BUFFER

```python
def forward(self, idx, targets=None, past_kvs=None, use_cache=False):
    device = idx.device
    b, t = idx.size()
    past_len = past_kvs[0][0].size② if past_kvs is not None and past_kvs[0] is not None else 0
    pos = torch.arange(past_len, past_len + t, dtype=torch.long, device=device)

    tok_emb = self.transformer.wte(idx)         # (B, T, C)
    pos_emb = self.transformer.wpe(pos)         # (T, C)
    x = self.transformer.drop(tok_emb + pos_emb)

    next_kvs = [] if use_cache else None
    for i, block in enumerate(self.transformer.h):
        past_kv = past_kvs[i] if past_kvs is not None else None
        x, next_kv = block(x, past_kv=past_kv, use_cache=use_cache)
        if use_cache:
            next_kvs.append(next_kv)

    x = self.transformer.ln_f(x)
    logits = self.lm_head(x)                    # (B, T, V)
    loss = F.cross_entropy(logits.view(-1, V), targets.view(-1)) if targets is not None else None
    return logits, loss, next_kvs
```

Math of position embeddings: When using cache, positions must account for past tokens. If cache has 100 tokens and we feed 5 new ones, positions are `[100, 101, 102, 103, 104]`.

Cross-entropy loss: `L = -Σ log softmax(logits)[target]` averaged over `B × T` predictions.

OPTIMIZER CONFIGURATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
def configure_optimizers(self, weight_decay, learning_rate, betas, device_type):
    decay, no_decay = set(), set()
    whitelist = (torch.nn.Linear,)
    blacklist = (torch.nn.LayerNorm, LayerNorm, torch.nn.Embedding)
    for mn, m in self.named_modules():
        for pn, p in m.named_parameters():
            fpn = f"{mn}.{pn}" if mn else pn
            if pn.endswith("bias"):
                no_decay.add(fpn)               # biases: no decay
            elif pn.endswith("weight") and isinstance(m, whitelist):
                decay.add(fpn)                  # linear weights: decay
            elif pn.endswith("weight") and isinstance(m, blacklist):
                no_decay.add(fpn)               # norms/embeddings: no decay
    if self.config.tie_weights:
        decay.discard("lm_head.weight")
    # ...create AdamW with two param groups
```

Math (AdamW):

[CODE // CODE] >> SOURCE BUFFER

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\theta_t = \theta_{t-1} - lr \cdot m_t / (\sqrt{v_t} + \varepsilon) - lr \cdot \lambda \cdot \theta_{t-1}$$

Why separate weight decay groups: Biases and LayerNorm params don't need regularization (they're already constrained). Applying weight decay to them hurts performance.

Why `lm_head.weight` **excluded:** It's tied to `wte.weight`, which is an embedding (no decay).

GENERATION WITH KV-CACHE

[CODE // PYTHON] >> SOURCE BUFFER

```python
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None, use_cache=True):
    self.eval()
    past_kvs = None
    for _ in range(max_new_tokens):
        idx_cond = idx if idx.size① <= self.config.block_size else idx[:, -self.config.block_size:]
        if use_cache:
            if past_kvs is None:
                logits, _, past_kvs = self(idx_cond, use_cache=True)
            else:
                logits, _, past_kvs = self(idx_cond[:, -1:], past_kvs=past_kvs, use_cache=True)
        else:
            logits, _, _ = self(idx_cond, use_cache=False)
        logits = logits[:, -1, :]
        if temperature == 0.0:
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)
        else:
            logits = logits / temperature
            if top_k is not None:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = -float("Inf")
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

Math of temperature: `softmax(logits/T)`. As `T → 0`, distribution becomes one-hot (greedy). As `T → ∞`, becomes uniform.

Math of top-k: Keep only top `k` logits, set rest to `-∞`. After softmax, only those `k` have nonzero probability.

KV-cache logic: First iteration processes full prompt, subsequent iterations process only the last token.

MODULE 4: MODEL CONFIGURATIONS — TARGET VS DRAFT

TARGET MODEL (10.8M)

```python
target_cfg = GPTConfig(
    block_size=256, vocab_size=10000, n_layer=9,
    n_head=4, n_embd=272, bias=True, tie_weights=True
)
```

Parameter breakdown: - Token embedding (tied): 10000 × 272 = 2,720,000 - Position embedding: 256 × 272 = 69,632 - Per block: - `c_attn`: 272 × 816 + 816 = 222,768 - `c_proj`: 272 × 272 + 272 = 74,256 - `c_fc`: 272 × 1088 + 1088 = 296,960 - `c_proj` (MLP): 1088 × 272 + 272 = 296,208 - 2 × LayerNorm: 2 × 544 = 1,088 - **Total per block: ~891,280** - 9 blocks: ~8,021,520 - Final LayerNorm: 544 - **Total: ~10,812,272** ✓

DRAFT MODEL (1.0M)

```python
draft_cfg = GPTConfig(
    block_size=256, vocab_size=10000, n_layer=1,
    n_head=2, n_embd=88, bias=True, tie_weights=True
)
```

Parameter breakdown: - Token embedding (tied): 10000 × 88 = 880,000 - Position embedding: 256 × 88 = 22,528 - 1 block: ~94,000 - **Total: ~996,776** ✓

Capacity asymmetry: 10.8M / 1.0M ≈ 10.9x

Design rationale: Draft model needs to be: - Fast enough that `γ` draft steps cost less than 1 target step - Accurate enough that acceptance rate is high (>65%) - Same tokenizer/vocab so distributions are comparable

MODULE 5: TINYSTORIES DATASET & BPE TOKENIZER

LOADING THE DATASET

```python
dataset = load_dataset("roneneldan/TinyStories", split="train", streaming=True)
num_stories = 30000
stories = []
for i, item in enumerate(dataset):
    stories.append(item["text"])
    if i + 1 >= num_stories:
        break
```

Why TinyStories: Simple vocabulary, coherent grammar. Small models can learn it well (unlike full internet text).

Why streaming: Avoids downloading the full ~2GB dataset. We only need 30K stories.

BPE TOKENIZER TRAINING

[CODE // PYTHON] >> SOURCE BUFFER

```python
from tokenizers import ByteLevelBPETokenizer

bpe = ByteLevelBPETokenizer()
bpe.train_from_iterator(
    train_stories,
    vocab_size=10000,
    min_frequency=2,
    special_tokens=["<|endoftext|>"]
)
```

Math (BPE algorithm): 1. Start with all 256 bytes as base vocab 2. Count adjacent pair frequencies across corpus 3. Merge most frequent pair into a new token 4. Repeat until vocab_size reached

Result: Subword tokenizer. `"playing"` might become `["play", "ing"]`. More efficient than character-level.

Why 10000: Balance between sequence length (smaller vocab → longer sequences) and embedding table size (larger vocab → more params).

ENCODING TO BINARY

[CODE // PYTHON] >> SOURCE BUFFER

```python
def encode_stories(stories_list, out_path):
    tokens = []
    eot = bpe.token_to_id("<|endoftext|>")
    for s in stories_list:
        tokens.extend(bpe.encode(s).ids)
        tokens.append(eot)  # document separator
    arr = np.array(tokens, dtype=np.uint16)
    arr.tofile(out_path)
    return arr
```

Math: Concatenate all token IDs into one long array. `uint16` because vocab_size=10000 < 65536.

`◁endoftext▷`: Special token separating documents. Model learns "story boundaries."

MODULE 6: DATALOADER & TRAINING PIPELINE

DATALOADER

[CODE // PYTHON] >> SOURCE BUFFER

```python
class DataLoader:
    def __init__(self, data_dir, block_size, batch_size, device):
        self.train_data = np.memmap(os.path.join(data_dir, "train.bin"), dtype=np.uint16, mode="r")
        self.val_data = np.memmap(os.path.join(data_dir, "val.bin"), dtype=np.uint16, mode="r")

    def get_batch(self, split="train"):
        data = self.train_data if split == "train" else self.val_data
        ix = torch.randint(len(data) - self.block_size, (self.batch_size,))
        x = torch.stack([torch.from_numpy((data[i : i + self.block_size]).astype(np.int64)) for i in ix])
        y = torch.stack([torch.from_numpy((data[i + 1 : i + 1 + self.block_size]).astype(np.int64)) for i in ix])
        return x.pin_memory().to(self.device, non_blocking=True), y.pin_memory().to(self.device, non_blocking=True)
```

Math: - `torch.randint(a, (b,))` : `b` random integers in `[0, a)` - `x` : `(B, T)` with `x[i, j] = data[pos_i + j]` - `y` : `(B, T)` with `y[i, j] = data[pos_i + j + 1]` (target is input shifted by 1)

`pin_memory` + `non_blocking` : Allows async CPU→GPU transfer while GPU computes.

COSINE LR SCHEDULE

[CODE // PYTHON] >> SOURCE BUFFER

```python
def get_cosine_lr(step, lr, warmup_iters, max_iters, min_lr):
    if step < warmup_iters:
        return lr * (step + 1) / warmup_iters       # linear warmup
    if step > max_iters:
        return min_lr
    ratio = (step - warmup_iters) / (max_iters - warmup_iters)
    return min_lr + 0.5 * (1.0 + math.cos(math.pi * ratio)) * (lr - min_lr)
```

Math: - Warmup: `lr_t = lr · t/T_warmup` for `t < T_warmup` - Cosine: `lr_t = lr_min + 0.5(1 + cos(π · (t-T_w)/ (T_max-T_w))) · (lr - lr_min)`

Shape: Rises linearly from 0 to `lr` over warmup, then decays smoothly to `lr_min` via cosine curve.

Why: Warmup prevents early training instability. Cosine decay improves final convergence.

EVALUATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
@torch.no_grad()
def evaluate_loss(model, dataloader, eval_iters=30):
    model.eval()
    losses = {"train": 0.0, "val": 0.0}
    for split in ["train", "val"]:
        total = 0.0
        for _ in range(eval_iters):
            x, y = dataloader.get_batch(split)
            _, loss, _ = model(x, targets=y)
            total += loss.item()
        losses[split] = total / eval_iters
    model.train()
    return losses
```

Math: Average cross-entropy over 30 batches. Reduces variance from single-batch estimates.

MODULE 7: TRAINING THE TARGET MODEL

TRAINING LOOP

[CODE // PYTHON] >> SOURCE BUFFER

```python
target_model = GPT(target_cfg).to(device)
dataloader = DataLoader("data/tinystories", block_size=256, batch_size=32, device=device)

optimizer = target_model.configure_optimizers(
    weight_decay=0.1, learning_rate=6e-4, betas=(0.9, 0.95), device_type=device
)
scaler = torch.amp.GradScaler(device=device, enabled=(device == "cuda"))
target_max_iters = 3000

for step in range(target_max_iters):
    lr = get_cosine_lr(step, 6e-4, 300, target_max_iters, 6e-5)
    for pg in optimizer.param_groups:
        pg["lr"] = lr

    if step % eval_interval == 0 or step == target_max_iters - 1:
        losses = evaluate_loss(target_model, dataloader)
        # ... save best checkpoint

    optimizer.zero_grad(set_to_none=True)
    x, y = dataloader.get_batch("train")
    with torch.amp.autocast(device_type=device, dtype=torch.float16):
        _, loss, _ = target_model(x, targets=y)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(target_model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```

Mixed precision (AMP) math: - Forward/backward in `float16`: 2× faster, half memory - Loss scaling: multiply loss by scale factor `S` to prevent underflow in fp16 gradients - `scaler.step`: divide gradients by `S` before applying update

Gradient clipping math:

```
[CODE // CODE]                                    >> SOURCE BUFFER

if ||g||₂ > 1.0:
    g ← g · 1.0 / ||g||₂
```

Prevents exploding gradients.

Hyperparameters: - `lr = 6e-4`: Standard for small Transformers (GPT-2 used 6e-4) - `weight_decay = 0.1`: Standard L2 regularization - `betas = (0.9, 0.95)`: Standard for Transformers (β_2=0.95 instead of 0.999 for stability) - `warmup = 300`: ~10% of total iterations - `batch_size = 32`, `block_size = 256`: 8192 tokens per step

MODULE 8: KNOWLEDGE DISTILLATION FOR DRAFT MODEL

WHY DISTILLATION?

The draft model isn't trained on raw text — it's trained to **mimic the target model's probability distribution**. This maximizes the acceptance rate.

THE LOSS

```
[CODE // PYTHON]                                  >> SOURCE BUFFER

logits, loss_ce, _ = draft_model(x, targets=y)
with torch.no_grad():
    t_logits, _, _ = target_model(x)

T_kd = 2.0
p_target = F.softmax(t_logits / T_kd, dim=-1)
log_p_draft = F.log_softmax(logits / T_kd, dim=-1)
loss_kd = F.kl_div(log_p_draft, p_target, reduction="batchmean") * (T_kd ** 2)
loss = 0.3 * loss_ce + 0.7 * loss_kd
```

Math (KL divergence):

```
[CODE // CODE]                                    >> SOURCE BUFFER

KL(p || q) = Σ p(x) log(p(x) / q(x))
```

Where `p = softmax(t_logits / T)` (teacher) and `q = softmax(draft_logits / T)` (student).

Temperature scaling: Dividing logits by `T=2` softens both distributions. This provides richer gradient signal (non-zero probabilities for all tokens, not just the argmax).

Gradient magnitude: `∂/∂z KL(softmax(z/T) || p) = (1/T)(softmax(z/T) - p)`. Multiplying loss by `T²` restores original gradient scale.

Combined loss: `0.3 × CE + 0.7 × KL`. The CE term keeps the draft grounded in true labels; the KL term aligns it with the target.

Why 70% weight on KL: The goal is distribution matching, not just next-token prediction. Higher KL weight → higher acceptance rate.

DRAFT TRAINING LOOP

[CODE // PYTHON] >> SOURCE BUFFER

```python
draft_model = GPT(draft_cfg).to(device)
target_model.eval()  # Freeze teacher

draft_optimizer = draft_model.configure_optimizers(
    weight_decay=0.1, learning_rate=1e-3, betas=(0.9, 0.95), device_type=device
)
draft_max_iters = 2500

for step in range(draft_max_iters):
    lr = get_cosine_lr(step, 1e-3, 200, draft_max_iters, 1e-4)
    # ... update LR
    # ... evaluate
    draft_optimizer.zero_grad(set_to_none=True)
    x, y = dataloader.get_batch("train")
    with torch.amp.autocast(device_type=device, dtype=torch.float16):
        logits, loss_ce, _ = draft_model(x, targets=y)
        with torch.no_grad():
            t_logits, _, _ = target_model(x)
        # ... compute KD loss
    draft_scaler.scale(loss).backward()
    draft_scaler.unscale_(draft_optimizer)
    torch.nn.utils.clip_grad_norm_(draft_model.parameters(), 1.0)
    draft_scaler.step(draft_optimizer)
    draft_scaler.update()
```

Note: Higher LR (`1e-3`) than target because draft is smaller. Target is in `eval()` mode and gradients not computed (`torch.no_grad()`).

MODULE 9: SPECULATIVE DECODING ENGINE

KV-CACHE TRUNCATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
def truncate_kv_cache(past_kvs, target_len):
    if past_kvs is None:
        return None
    return [(k[:, :, :target_len, :], v[:, :, :target_len, :]) for k, v in past_kvs]
```

Math: After rejection at position `i`, the caches contain K/V for positions we no longer accept. Slice to `target_len` to remove them.

Cache shape: `(B, H, T_cache, D)` → slice `dim=2` to `(B, H, target_len, D)`.

MAIN GENERATION LOOP

[CODE // PYTHON] >> SOURCE BUFFER

```python
class SpeculativeDecoder:
    def __init__(self, target_model, draft_model):
        self.target_model = target_model.eval()
        self.draft_model = draft_model.eval()

    @torch.inference_mode()
    def generate(self, prompt, max_new_tokens, gamma=3, temperature=0.0):
        seq = prompt.clone().to(self.device)
        prompt_len = seq.size①
        total_len = prompt_len + max_new_tokens

        # Prefill: process prompt once, get initial logits + caches
        target_logits, _, target_kvs = self.target_model(seq, use_cache=True)
        draft_logits, _, draft_kvs = self.draft_model(seq, use_cache=True)
        last_target_logits = target_logits[:, -1, :]
        last_draft_logits = draft_logits[:, -1, :]
        pending_target_token = None
```

Math of prefill: Process the entire prompt in one forward pass. Get: - Logits for all positions - KV cache for all layers - The "last" logits (for position `prompt_len`) predict the next token

THE MAIN LOOP

[CODE // PYTHON] >> SOURCE BUFFER

```python
while seq.size① < total_len:
    cycles += 1
    cur_gamma = min(gamma, total_len - seq.size①)
    prefix_len = seq.size①
```

Math: `cur_gamma` is γ normally, but less on the final cycle to not overshoot `total_len`.

PHASE 1: DRAFT GENERATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
draft_tokens = []
draft_probs = []
cur_logits = last_draft_logits

for _ in range(cur_gamma):
    if temperature == 0.0:
        tok = torch.argmax(cur_logits, dim=-1, keepdim=True)
        p = F.softmax(cur_logits, dim=-1)
    else:
        p = F.softmax(cur_logits / temperature, dim=-1)
        tok = torch.multinomial(p, 1)
    draft_tokens.append(tok)
    draft_probs.append(p)
    step_logits, _, draft_kvs = self.draft_model(tok, past_kvs=draft_kvs, use_cache=True)
    cur_logits = step_logits[:, -1, :]

cand_tokens = torch.cat(draft_tokens, dim=1)  # (1, γ)
```

Math: For each of `γ` steps: 1. Get probability `p` from current logits (softmax) 2. Sample `tok` from `p` (or argmax for greedy) 3. Feed `tok` through draft model (single token, with cache) 4. Update `cur_logits` to predict next token

Cost: `γ` sequential draft forward passes. Each is cheap (1-layer model).

PHASE 2: TARGET VERIFICATION (ONE FORWARD PASS)

[CODE // PYTHON] >> SOURCE BUFFER

```python
if pending_target_token is None:
    # First cycle: cache covers prompt
    t_cand_logits, _, target_kvs = self.target_model(cand_tokens, past_kvs=target_kvs, use_cache=True)
    all_target_logits = torch.cat([last_target_logits.unsqueeze①, t_cand_logits[:, :-1, :]], dim=1)
    bonus_logits = t_cand_logits[:, -1, :]
else:
    # Later cycles: prepend previous emitted token
    tokens_to_target = torch.cat([pending_target_token, cand_tokens], dim=1)
    t_cand_logits, _, target_kvs = self.target_model(tokens_to_target, past_kvs=target_kvs, use_cache=True)
    all_target_logits = t_cand_logits[:, :-1, :]
    bonus_logits = t_cand_logits[:, -1, :]
target_forward_passes += 1
```

Math — this is the key insight:

For cycle 1, the target cache already has K/V for `[prompt]`. We feed `[cand₁, cand₂, ..., cand_γ]` (γ tokens) → get γ+1 logits (including the initial `last_target_logits` for position `prompt_len`).

`all_target_logits[i]` = target distribution for predicting token `i+1` given `[prompt, cand₁, ..., candᵢ]`.

For cycle 2+, we need to account for the token we emitted last cycle (call it `pending`). We feed `[pending, cand₁, ..., cand_γ]` → γ+1 logits. `all_target_logits[i]` is now the distribution for the i-th position.

Critical: This is **ONE** target forward pass for all `γ` verifications. That's the entire speedup.

PHASE 3A: GREEDY ACCEPTANCE (VECTORIZED)

[CODE // PYTHON] >> SOURCE BUFFER

```python
if temperature == 0.0:
    tgt_greedy = torch.argmax(all_target_logits, dim=-1)  # (1, γ)
    matches = (cand_tokens == tgt_greedy)[0]              # (γ,)
    num_acc = torch.cumprod(matches.long(), dim=0).sum().item()

    if num_acc == cur_gamma:
        num_eval = cur_gamma
        rejected = False
        replacement = None
    else:
        num_eval = num_acc + 1
        rejected = True
        replacement = tgt_greedy[:, num_acc : num_acc + 1]
    accepted = [cand_tokens[:, :num_acc]] if num_acc > 0 else []
```

Math of `cumprod`:

[CODE // CODE] >> SOURCE BUFFER

```
matches = [1, 1, 0, 1]  (first rejection at index 2)
cumprod = [1, 1, 0, 0]  (everything after first rejection → 0)
sum = 2 = number of accepted
```

This counts consecutive matches from the start. **Must accept all tokens before the first rejection** — that's the correctness condition for greedy speculative decoding.

Acceptance logic (greedy): - If `cand_tokens[i] == argmax(target_logits[i])`, accept (draft made same greedy choice) - Else, reject and use target's argmax as replacement - After rejection, all subsequent candidates are discarded

`num_eval` = `num_acc + 1`: We evaluated `num_acc` accepted + 1 rejected = `num_acc + 1` candidates.

`replacement`: Target's greedy choice at the rejection position.

PHASE 3B: STOCHASTIC ACCEPTANCE (REJECTION SAMPLING)

[CODE // PYTHON] >> SOURCE BUFFER

```python
else:
    accepted = []
    rejected = False
    replacement = None
    num_eval = 0
    for i in range(cur_gamma):
        num_eval += 1
        cand = cand_tokens[:, i : i + 1]
        p_d = draft_probs[i][0, cand.item()].item()
        t_p = F.softmax(all_target_logits[:, i, :] / temperature, dim=-1)
        p_t = t_p[0, cand.item()].item()
        if torch.rand(1, device=self.device).item() < min(1.0, p_t / max(p_d, 1e-12)):
            accepted.append(cand)
        else:
            rejected = True
            res = torch.clamp(t_p - draft_probs[i], min=0.0)
            replacement = torch.multinomial(res / res.sum(), 1) if res.sum() > 0 else torch.multinomial(t_p, 1)
            break
    num_acc = len(accepted)
```

Math (Leviathan rejection sampling):

For each candidate $\tilde{x}_i$: 1. Get draft probability $p_d = p_draft(\tilde{x}_i)$ 2. Get target probability $p_t = p_target(\tilde{x}_i)$ 3. Accept with probability $\min(1, p_t / p_d)$

Why this works: If $p_t \geq p_d$, always accept. If $p_t < p_d$, accept with probability p_t/p_d. The expected acceptance rate at position i is $\Sigma \min(p_d(x), p_t(x))$.

Residual sampling (on rejection):

[CODE // CODE] >> SOURCE BUFFER

```
residual = max(0, p_target - p_draft)  # element-wise
replacement ~ residual / Σ(residual)
```

Why: This corrects for the fact that we rejected. Mathematically:

[CODE // CODE] >> SOURCE BUFFER

```
P(output = x) = P(accept x) + P(reject) · P(sample x | reject)
              = min(p_d(x), p_t(x)) + [1 - Σ min(p_d, p_t)] · max(0, p_t(x) - p_d(x)) / Σ max(0, p_t - p_d)
              = min(p_d(x), p_t(x)) + max(0, p_t(x) - p_d(x))
              = p_t(x) ✓
```

This is the theorem: Speculative decoding is exact. The output distribution is p_target.

PHASE 4: CACHE MANAGEMENT

[CODE // PYTHON] >> SOURCE BUFFER

```python
if not rejected:
    # All γ candidates accepted; emit bonus token
    bonus = torch.argmax(bonus_logits, dim=-1, keepdim=True) if temperature == 0.0 \
        else torch.multinomial(F.softmax(bonus_logits / max(temperature, 1e-6), dim=-1), 1)
    seq = torch.cat([seq, cand_tokens, bonus], dim=1)
    pending_target_token = bonus
    d_step, _, draft_kvs = self.draft_model(bonus, past_kvs=draft_kvs, use_cache=True)
    last_draft_logits = d_step[:, -1, :]
else:
    emitted = torch.cat([cand_tokens[:, :num_acc], replacement], dim=1) if num_acc > 0 else replacement
    seq = torch.cat([seq, emitted], dim=1)
    valid_len = prefix_len + num_acc
    target_kvs = truncate_kv_cache(target_kvs, valid_len)
    draft_kvs = truncate_kv_cache(draft_kvs, valid_len)
    pending_target_token = replacement
    d_step, _, draft_kvs = self.draft_model(replacement, past_kvs=draft_kvs, use_cache=True)
    last_draft_logits = d_step[:, -1, :]
```

All accepted case: - Append `[cand₁, ..., cand_γ, bonus]` to sequence - `bonus` = target's greedy/sampled choice at position after `cand_γ` (free extra token!) - Update draft cache with `bonus` (draft was not run on it) - Set `pending_target_token = bonus` for next cycle

Rejected case: - Append `[cand₁, ..., cand_num_acc, replacement]` - Truncate both caches to `prefix_len + num_acc` - Update draft cache with `replacement` - Set `pending_target_token = replacement`

Cache synchronization: After truncation, both caches contain K/V for positions `[0, prefix_len + num_acc)`. Next cycle, target processes `[pending, cand₁, ..., cand_γ]` and appends to cache.

Why `pending_target_token`: The last emitted token's K/V is not yet in the target cache (target was run on candidates, not on the emitted token). Next cycle, we feed it in.

METRICS

[CODE // PYTHON] >> SOURCE BUFFER

```python
return final_seq, {
    "elapsed": elapsed,
    "tps": gen_tokens / elapsed,
    "acceptance_rate": draft_accepted / max(draft_evaluated, 1),
    "slot_efficiency": draft_accepted / max(draft_proposed, 1),
    "cycles": cycles,
    "gen_tokens": gen_tokens,
    "target_forward_passes": target_forward_passes
}
```

Acceptance rate: `accepted / evaluated`. Numerator is tokens actually kept; denominator is tokens tested.

Slot efficiency: `accepted / proposed`. Measures what fraction of draft capacity was useful.

Speedup vs baseline: `baseline_time / speculative_time`.

Theoretical speedup model: If `α` is acceptance rate, `γ` is speculation length, `c` is draft/target cost ratio:

[CODE // CODE] >> SOURCE BUFFER

```
Speedup ≈ (1 - α^(γ+1)) / ((1 - α)(γc + 1))
```

For α=0.65, γ=3, c=0.1: `(1 - 0.65⁴) / (0.35 × 1.3) = 0.821 / 0.455 ≈ 1.80x`

MODULE 10: BENCHMARKING

SETUP

[CODE // PYTHON] >> SOURCE BUFFER

```python
decoder = SpeculativeDecoder(target_model, draft_model)

benchmark_prompts = [
    "Once upon a time, there was a little girl named Lily who had a cat.",
    ...
]

# GPU Warm-Up (critical for accurate timing)
warmup_ids = torch.tensor([[100, 200, 300]], dtype=torch.long, device=device)
_ = target_model.generate(warmup_ids, max_new_tokens=10, temperature=0.0, use_cache=True)
_ = decoder.generate(warmup_ids, max_new_tokens=10, gamma=3, temperature=0.0)
torch.cuda.synchronize()
```

Why warmup: CUDA kernel compilation, cuBLAS handle initialization, memory allocation all happen on first call. Must exclude from measurement.

`torch.cuda.synchronize()`: Ensures all async GPU operations complete before timing.

PER-PROMPT TIMING

[CODE // PYTHON] >> SOURCE BUFFER

```python
for idx, prompt_text in enumerate(benchmark_prompts):
    p_ids = bpe.encode(prompt_text).ids
    p_tensor = torch.tensor([p_ids], dtype=torch.long, device=device)

    # Baseline
    torch.cuda.synchronize()
    t0 = time.perf_counter()
    _ = target_model.generate(p_tensor, max_new_tokens=100, temperature=0.0, use_cache=True)
    torch.cuda.synchronize()
    base_time = time.perf_counter() - t0

    # Speculative
    _, stats = decoder.generate(p_tensor, max_new_tokens=100, gamma=3, temperature=0.0)
    spec_time = stats["elapsed"]
    speedup = base_time / max(spec_time, 1e-6)
```

Math: `time.perf_counter()` measures wall-clock time. `torch.cuda.synchronize()` ensures the GPU work is complete before we stop timing (otherwise we'd measure only kernel launch time, not execution time).

100 tokens is short enough to run quickly but long enough to average out noise.

MODULE 11: INTERACTIVE GENERATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
custom_prompt = "One afternoon, Leo found a shiny golden key hidden under a rock."
p_ids = bpe.encode(custom_prompt).ids
p_tensor = torch.tensor([p_ids], dtype=torch.long, device=device)

out_tokens, stats = decoder.generate(p_tensor, max_new_tokens=80, gamma=4, temperature=0.7)
story_text = bpe.decode(out_tokens[0].tolist())
```

Math of sampling with `temperature=0.7` **:**

[CODE // CODE] >> SOURCE BUFFER

```
p(x) = softmax(logits / 0.7)
```

Lower than 1.0 → sharper distribution → more focused output. Still stochastic.

Gamma=4: Speculate 4 tokens per cycle. Trade-off: - Higher γ → more tokens per target pass → more speedup potential - Higher γ → more likely to reject later candidates → wasted draft compute - Optimal γ typically 3-5 for well-aligned draft models

MODULE 12: VISUALIZATION

[CODE // PYTHON] >> SOURCE BUFFER

```python
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Loss curves
axs[0].plot(t_steps, t_val, label="Target Model (10.8M) Val Loss")
axs[0].plot(d_steps, d_val, label="Draft Model (1.0M) Val Loss", linestyle="--")

# Speedup vs acceptance rate
ax1.bar(labels, speedups, color="tab:blue", alpha=0.6, label="Speedup (x)")
ax1.axhline(2.1, color="navy", linestyle="--", label="Target Goal (2.1x)")

ax2 = ax1.twinx()
ax2.plot(labels, acc_rates, color="tab:orange", marker="o", label="Acceptance Rate (%)")
ax2.axhline(65, color="darkorange", linestyle=":", label="Target Goal (65%)")
```

Dual-axis plot: Left axis = speedup (bar chart), right axis = acceptance rate (line chart). Shows correlation between the two.

SUMMARY TABLE: EVERY COMPONENT

COMPONENT	INPUT SHAPE	OUTPUT SHAPE	MATH OPERATION
Token Embedding	(B, T)	(B, T, C)	Lookup
Position Embedding	(T,)	(T, C)	Lookup
Addition	(B,T,C) + (T,C)	(B,T,C)	Broadcast add
LayerNorm	(B, T, C)	(B, T, C)	`(x-μ)/σ · γ + β`
Q,K,V Projection	(B, T, C)	(B, T, 3C)	`x @ W_qkv^T`
Split + Reshape	(B, T, 3C)	3×(B, H, T, D)	Split, view, transpose
KV-Cache Concat	(B,H,T_new,D) + (B,H,T_past,D)	(B,H,T_total,D)	`cat(dim=2)`
QK^T	(B,H,T,D) @ (B,H,D,T_total)	(B,H,T,T_total)	Matmul
Scale	(B,H,T,T_total)	same	`× 1/√D`
Causal Mask	(B,H,T,T_total)	same	`masked_fill(-inf)`
Softmax	(B,H,T,T_total)	same	Row-wise softmax
Attention @ V	(B,H,T,T_total) @ (B,H,T_total,D)	(B,H,T,D)	Matmul
Output Projection	(B, T, C)	(B, T, C)	`x @ W_o^T`
MLP	(B, T, C)	(B, T, C)	$\text{GELU}(xW_1)W_2$
Residual Add	(B,T,C) + (B,T,C)	(B,T,C)	Add
LM Head	(B, T, C)	(B, T, V)	`x @ W_lm^T`
Cross-Entropy	(B·T, V), (B·T,)	scalar	`-log softmax[target]`
KD Loss	(B·T, V), (B·T, V)	scalar	`KL(softmax(t/T) || softmax(s/T))·T²`

KEY MATHEMATICAL INSIGHTS

1. **Speculative decoding is exact:** Rejection sampling with `α = min(1, q/p)` followed by residual resampling from `max(0, q-p)` yields the exact target distribution `q`.

2. **One target forward pass per cycle:** Because the Transformer is causal, feeding `[t₁, ..., t_γ]` gives `γ+1` predictions in one matmul. This is the entire speedup mechanism.
3. **KV-cache is essential:** Without it, verification would require `γ` sequential target passes, negating the speedup.
4. **Distillation maximizes acceptance:** Training the draft to match the target's *distribution* (not just argmax) increases `Σ min(p_d, p_t)`, the expected acceptance rate.
5. **Acceptance rate formula:** `P(accept) = Σ_x min(p_d(x), p_t(x))`. For greedy decoding, this becomes `P(argmax_d = argmax_t)`.
6. **Theoretical speedup:** `(1 - α^(γ+1)) / ((1-α)(γc + 1))` where `c` is the draft/target cost ratio.
7. **Greedy vectorization:** `cumprod(matches)` gives a mask for tokens up to the first mismatch. `sum()` gives the count. This avoids a Python loop.
8. **Cache rollback:** After rejection at position `i`, truncate both caches to `prefix_len + i` and feed the replacement token to the draft.
9. **Weight tying:** `wte.weight = lm_head.weight` saves `V × C` params and improves performance.
10. **Scaled residual init:** `std = 0.02 / √(2L)` for output projections prevents variance explosion in deep networks.